



CAPSTONE PROJECT: SERVERLESS TRANSLATION PIPELINE WITH AWS TRANSLATE AND AMAZON S3

INTRODUCTION

The objective of this capstone project was to design, develop, and implement an Infrastructure-as-Code (IaC) solution on AWS that automates real-time language translation. The solution integrates Amazon S3, AWS Lambda, and Amazon Translate with infrastructure provisioned using Terraform. The system is designed to remain fully serverless and operate within AWS Free Tier limits when used with small test data.

PROJECT GOALS

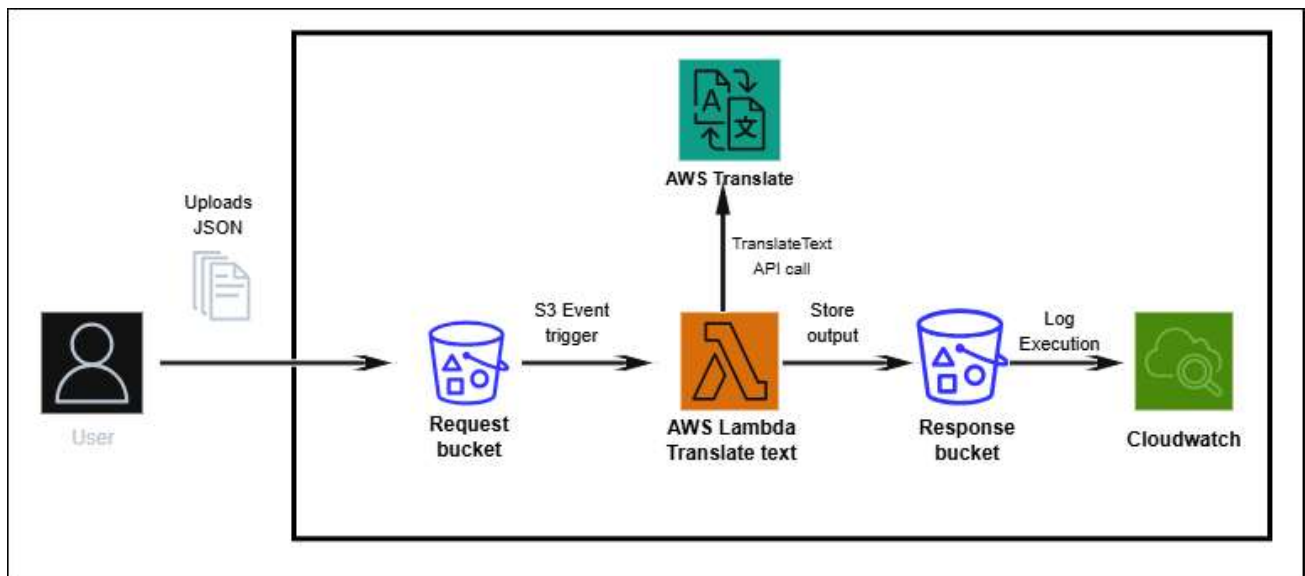
- Automate translation of text from uploaded JSON files.
- Store original requests and translated responses securely in Amazon S3.
- Ensure scalability and low operational overhead using serverless architecture.
- Deploy infrastructure reproducibly with Terraform (IaC).
- Enforce best practices for IAM, CloudWatch logging, and cost efficiency.

ARCHITECTURE OVERVIEW

The solution follows an event driven pipeline:

Component	Purpose
Request S3 Bucket	Stores incoming JSON files with translation requests.
S3 Event Trigger	When a file is uploaded, it triggers the Lambda function.
Lambda Function (Python, Boto3)	Reads the file, calls Amazon Translate, and writes results.
Response S3 Bucket	Stores translated JSON files.
CloudWatch Logs	Captures execution logs and errors for debugging.
Terraform	Automates provisioning of S3 buckets, IAM roles, and Lambda function.

ARCHITECTURAL DIAGRAM



IMPLEMENTATION DETAILS

PHASE 1: Initial Setup & AWS configuration

The first phase focused on laying the foundation for the translation pipeline by setting up the necessary AWS resources, permissions, and development environment.

I. Researched Amazon Translate (real-time neural machine translation service) and Amazon S3 (object storage).

- Explored the capabilities of Amazon Translate, including supported languages, text size limits (5,000 bytes per request), and free-tier usage (2M characters/month for 12 months).

- Studied Amazon S3 features such as versioning, encryption, bucket policies, lifecycle rules, and event notifications, which would later be used for automation.

II. Installed and configured required tools:

- AWS CLI (for local AWS interaction).
- Terraform (for Infrastructure-as-Code provisioning).
- Python 3.9+ (for Lambda function development).

III. Configured AWS credentials locally using *aws configure*.

IV. Drafted IAM policy with scoped access:

- translate:TranslateText (Amazon Translate).
- s3:GetObject, s3:PutObject, s3:ListBucket (Amazon S3).
- CloudWatch logging permissions for Lambda.

Tools: AWS CLI, IAM, Amazon S3.

Free Tier Compliance: S3 (5 GB), IAM (Always Free)

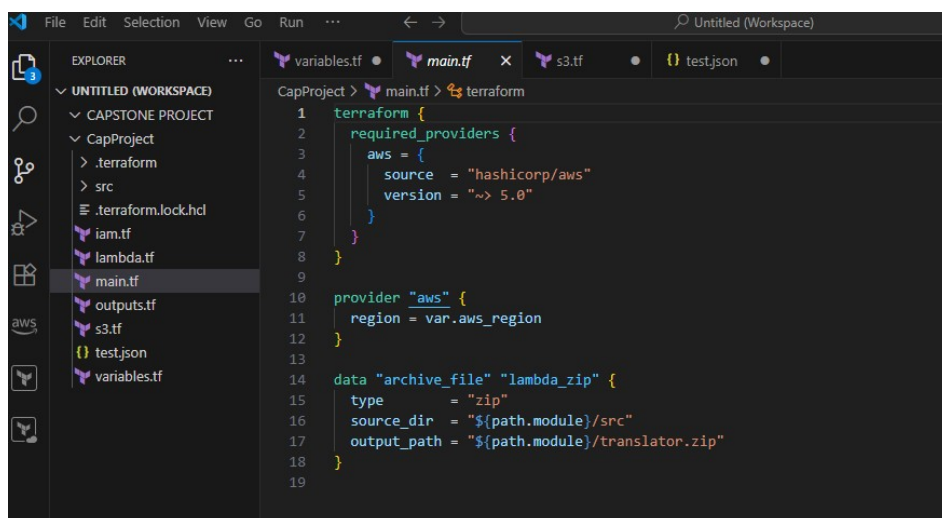
PHASE 2: Infrastructure-as-Code (IaC) Design

In this phase, Terraform was used to fully automate the provisioning of AWS resources, including S3 buckets, IAM roles, and the Lambda deployment. This ensured a reproducible and error-free setup compared to manual configuration.

ACTIVITIES

A. Created Terraform provider configuration:

To begin infrastructure provisioning, a Terraform provider block was created to define the AWS region and authentication method. This configuration ensures that all resources are deployed consistently within the selected environment.



```

1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 5.0"
6     }
7   }
8 }
9
10 provider "aws" {
11   region = var.aws_region
12 }
13
14 data "archive_file" "lambda_zip" {
15   type = "zip"
16   source_dir = "${path.module}/src"
17   output_path = "${path.module}/translator.zip"
18 }
19

```

Figure 1: Terraform provider block specifying AWS region and authentication.

B. Defined variables for region, input/output buckets, and Lambda function:

To make the infrastructure modular and reusable, key parameters were abstracted into Terraform variables. This allowed for easy updates to bucket names, Lambda configuration, and region settings without modifying core resource blocks.

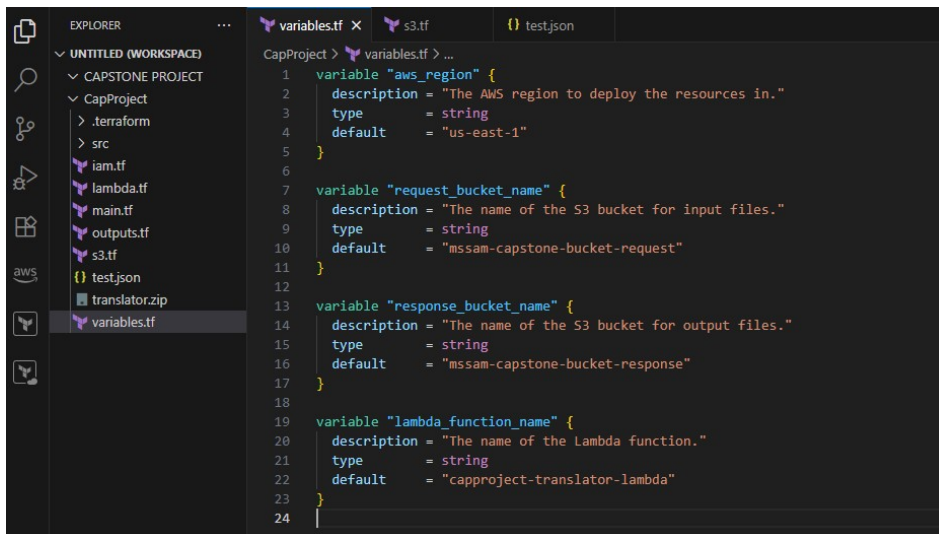


Figure 2: Terraform variable declarations for AWS region, S3 buckets, and Lambda function.

C. S3 Buckets with Lifecycle Policies

Two Amazon S3 buckets were provisioned using Terraform:

- request-bucket – to store incoming JSON files requiring translation.
- response-bucket – to store translated JSON files after processing.

Lifecycle rules were added to automatically transition or expire files after a set period, ensuring cost optimization within Free Tier limits.

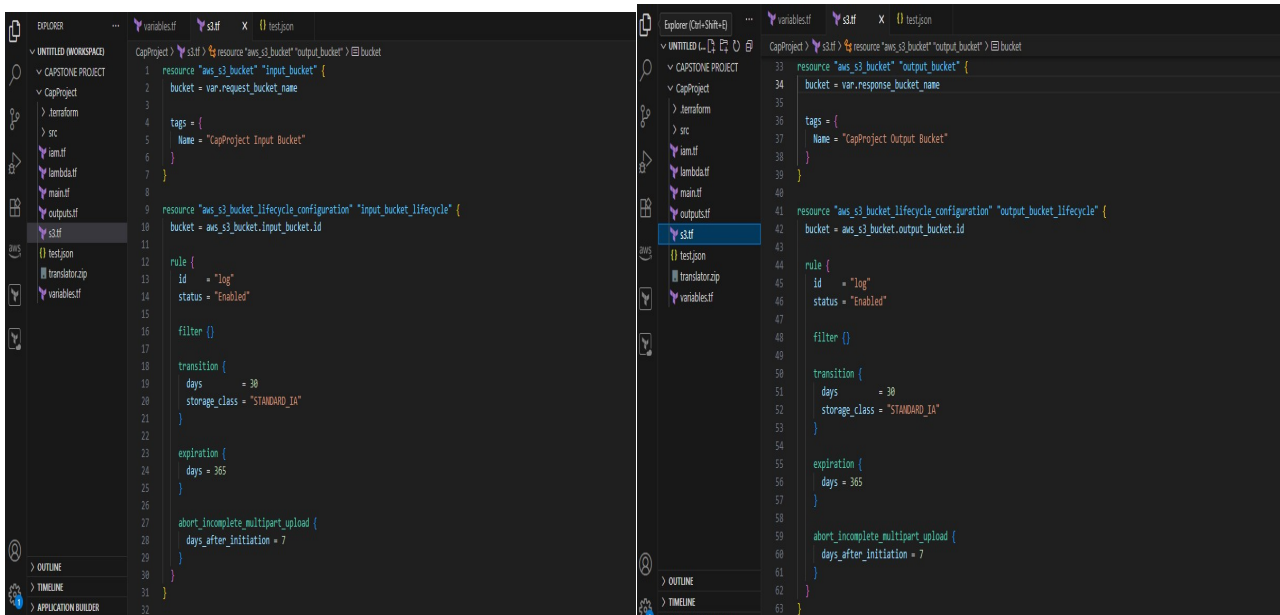


Figure 3 & 4 : Terraform resource block for S3 buckets with lifecycle configuration.

After provisioning the S3 buckets, Terraform outputs the names of the input and output buckets: This step ensures visibility into key resources and simplifies integration with other modules or scripts.

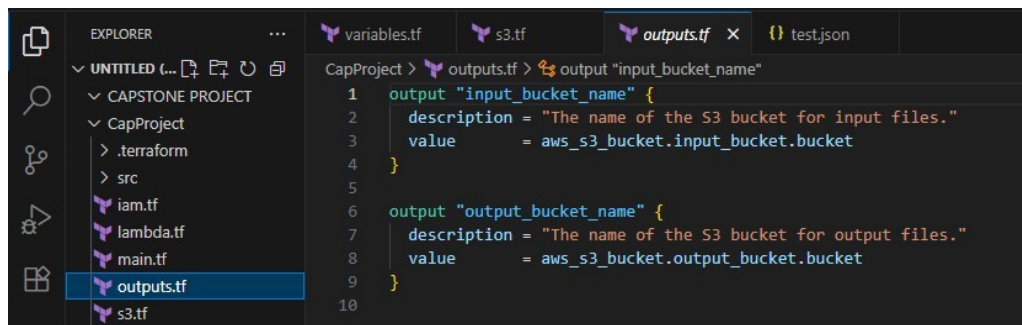


Figure 5: Terraform outputs.tf showing request and response bucket names.

D. IAM Role and Policy for Lambda Execution

To enforce least privilege and secure access, Terraform was used to define a scoped IAM role and policy for the Lambda function. This ensured the function could only interact with the specific AWS services required for translation, logging, and file handling.

- Amazon S3 – read access for input bucket, write access for output bucket.
- Amazon Translate – to perform text translations.
- Amazon Comprehend – to auto-detect source language.
- CloudWatch Logs – to capture logs for monitoring and debugging.

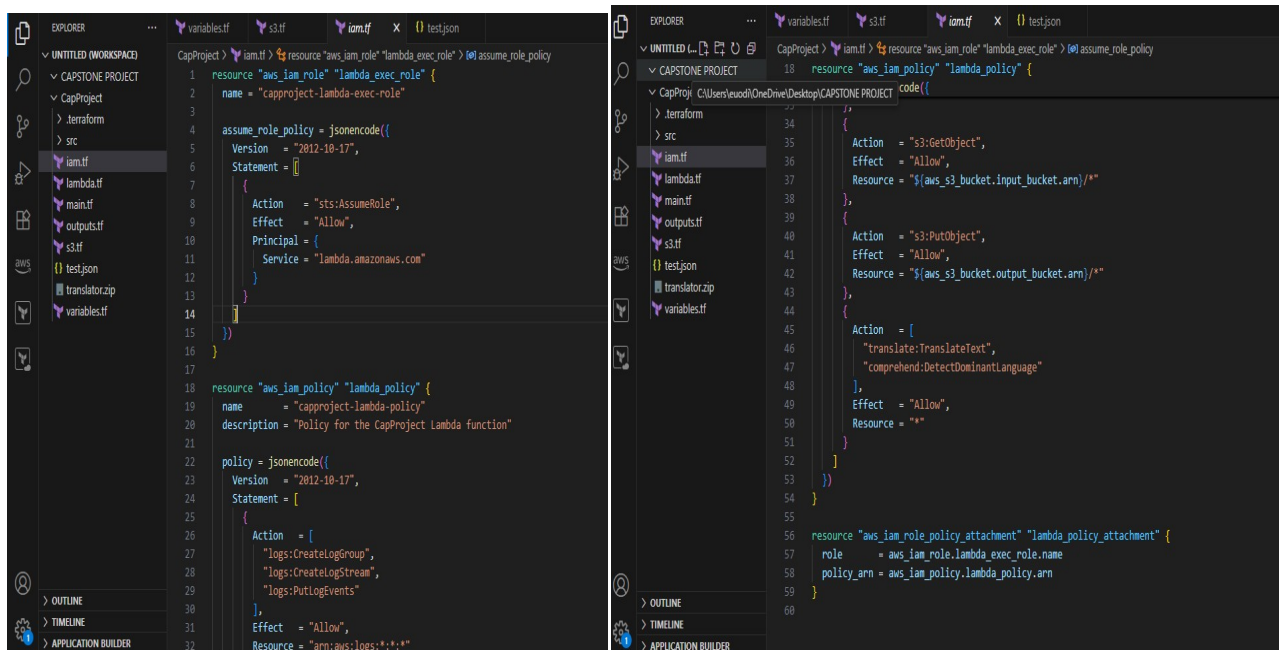


Figure 6 & 7: IAM role and policy definition for Lambda execution.

E. Validation

Once all Terraform configurations were written and validated, the deployment process was initiated using the following commands:

Ran *terraform init* and *terraform apply* to deploy stack.

- *terraform init* — Initialized the working directory and downloaded required providers


```

PS C:\Users\euodi\OneDrive\Desktop\CapProject> terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/archive from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0
- Using previously-installed hashicorp/archive v2.7.1

Terraform has been successfully initialized!

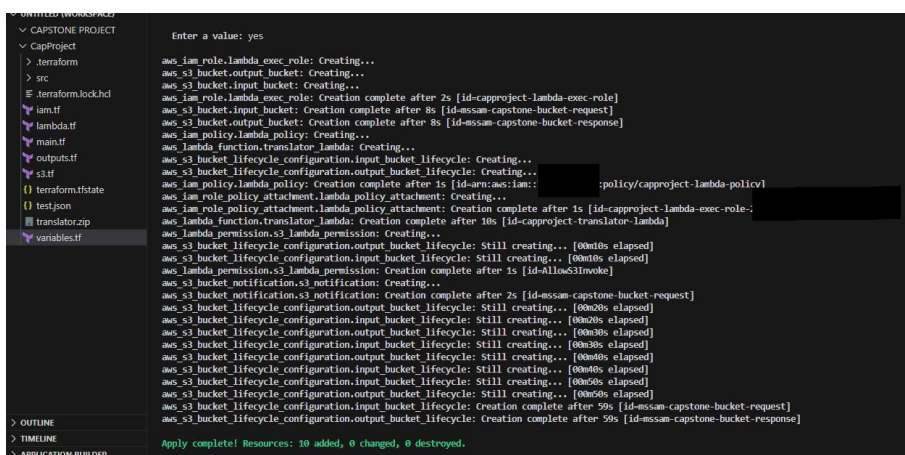
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

Figure 8: *terraform init*

- *terraform apply* — Executed the provisioning plan and created all defined AWS resources



```

Enter a value: yes
aws_iam_role.lambda_exec_role: Creating...
aws_s3_bucket.output_bucket: Creating...
aws_iam_role.lambda_exec_role: Creation complete after 2s [id=capproject-lambda-exec-role]
aws_s3_bucket.input_bucket: Creation complete after 8s [id=msam-capstone-bucket-request]
aws_s3_bucket.output_bucket: Creation complete after 8s [id=msam-capstone-bucket-response]
aws_iam_policy.lambda_policy: Creating...
aws_lambda_function.translator_lambda: Creating...
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Creating...
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Creating...
aws_iam_policy.lambda_policy: Creation complete after 1s [id=arn:aws:iam::[REDACTED]:policy/capproject-lambda-policy]
aws_iam_role_policy_attachment.lambda_policy_attachment: Creating...
aws_iam_role_policy_attachment.lambda_policy_attachment: Creation complete after 1s [id=capproject-lambda-exec-role-]
aws_lambda_function.translator_lambda: Creation complete after 10s [id=capproject-translator-lambda]
aws_lambda_permission.s3_lambda_permission: Creating...
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_lambda_permission.s3_lambda_permission: Creation complete after 1s [id=AllowS3Invoke]
aws_s3_bucket_notification.s3_notification: Creating...
aws_s3_bucket_notification.s3_notification: Creation complete after 2s [id=msam-capstone-bucket-request]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Still creating... [00m0s elapsed]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Creation complete after 59s [id=msam-capstone-bucket-request]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Creation complete after 59s [id=msam-capstone-bucket-response]

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

```

Figure 9: *terraform apply*

- After deploying the infrastructure with Terraform, each component was manually verified in the AWS Console to ensure correct configuration and integration. This step confirmed that the system was functioning as intended and ready for automated translation tasks.

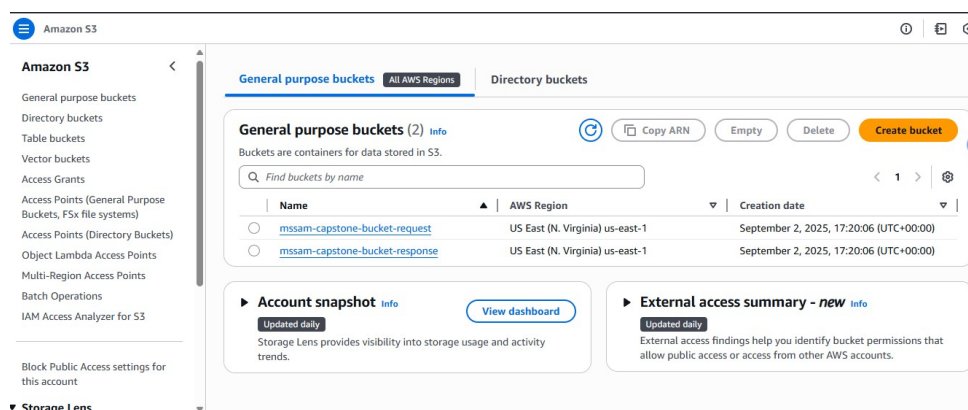


Figure 10: *Amazon S3 buckets configured for input and output storage*

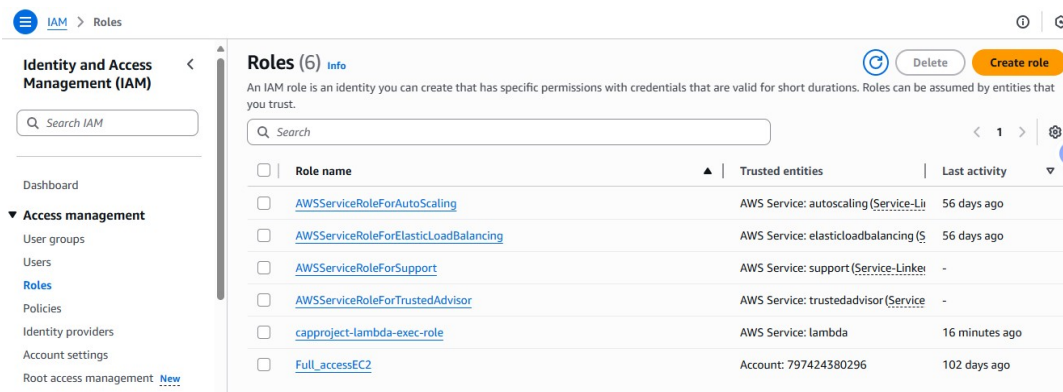


Figure 11: IAM role assigned to Lambda function

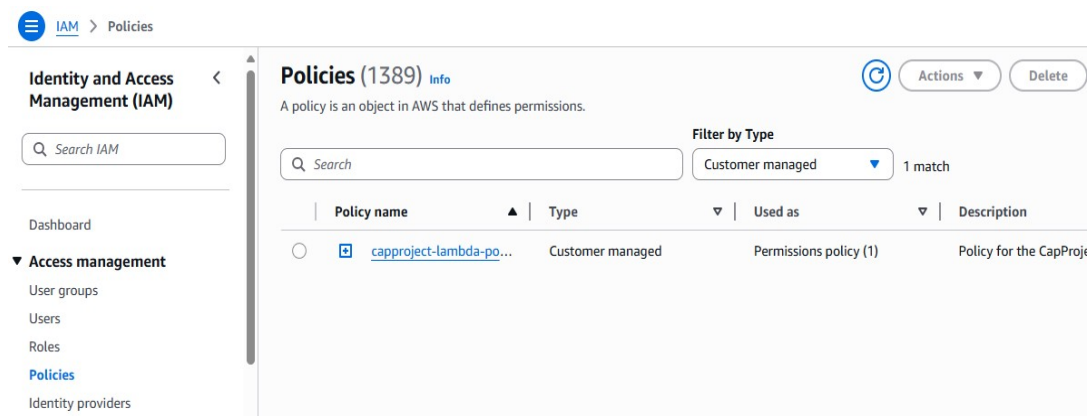


Figure 12: IAM policy assigned to Lambda function

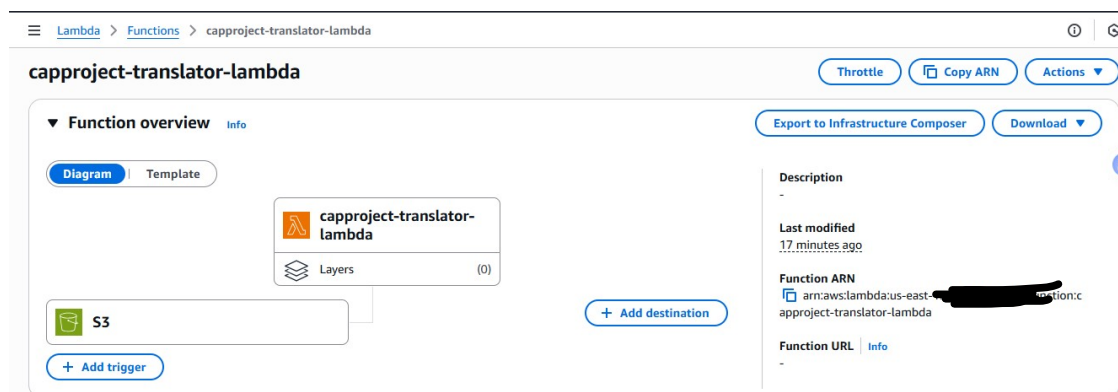


Figure 13: S3 event trigger linked to Lambda execution

Tools: Terraform, AWS IAM, Amazon S3

Free Tier Compliance: Terraform (No cost), S3 + IAM (within AWS free tier limits)

PHASE 3: Backend Script Development (Translation Logic)

ACTIVITIES

Developed Python script (Lambda handler) using Boto3:

The core of the translation pipeline was implemented in a Python script deployed as an AWS Lambda function. This handler uses the Boto3 SDK to interact with Amazon S3, Amazon Translate, and Amazon Comprehend.

- Reads input JSON file with text + target language.
- Uses Amazon Translate API.
- Stores translated JSON in response-bucket.

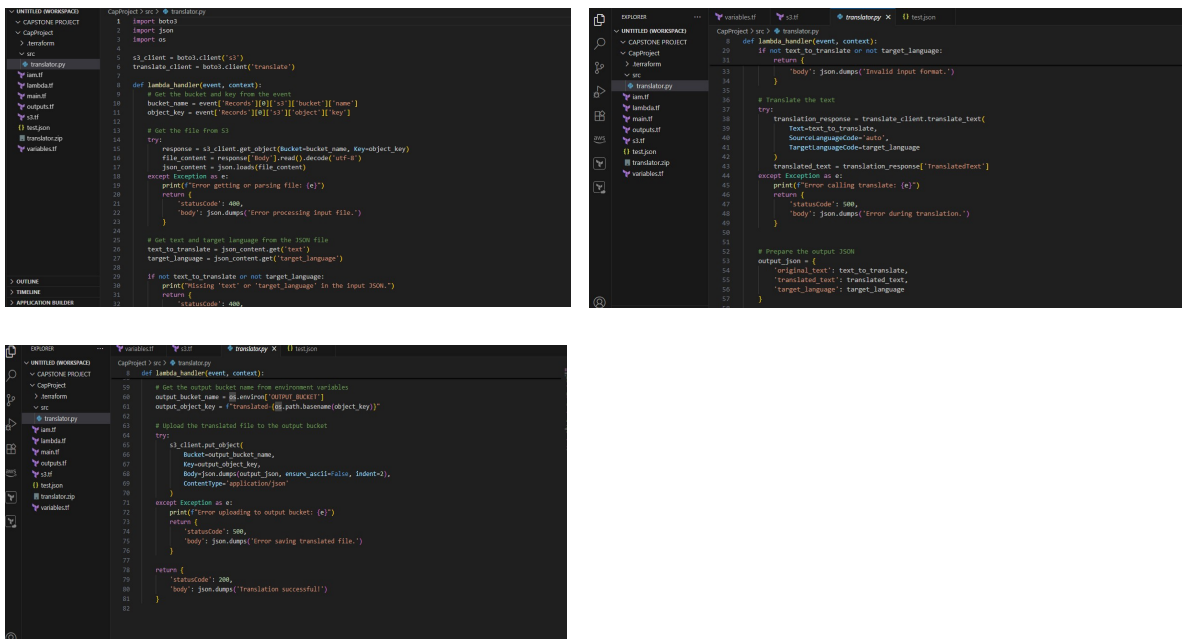


Figure 14, 15 & 16: Python script using Boto3 to read input from S3, translate text, and write output

- To initiate the translation process, a structured JSON file is uploaded to the request bucket. This file contains the text to be translated and the desired target language code.

JSON input:

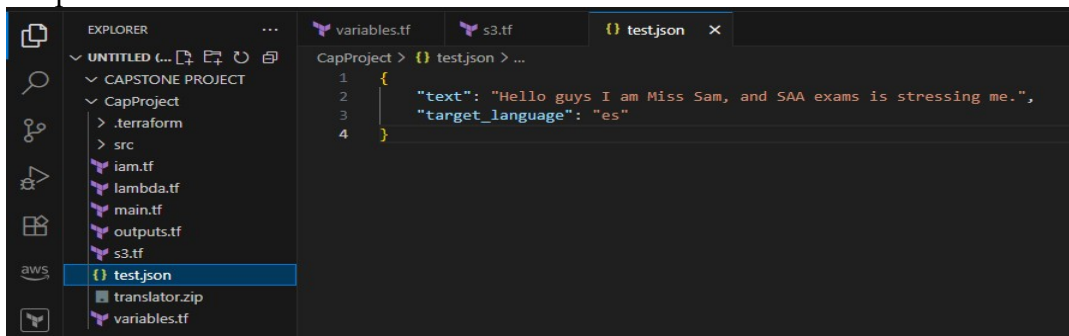
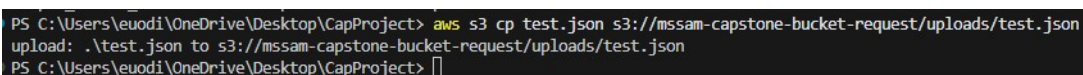


Figure 17: Sample JSON file uploaded to S3 for translation

- To trigger the translation pipeline, the sample JSON file was uploaded to the designated folder in the request bucket using the AWS CLI. This action initiates the event-driven flow by invoking the Lambda function automatically

Used this code to upload into request bucket:

aws s3 cp test.json s3://mssam-capstone-bucket-request/uploads/test.json



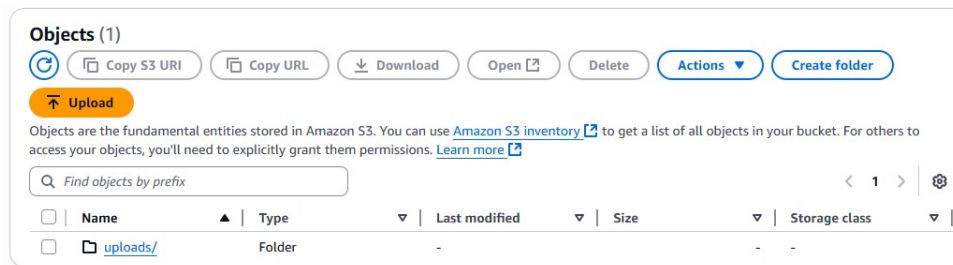


Figure 18 & 19: Uploading the test.json file into the request bucket.

- Translated file in the response bucket & translated json text:

After the Lambda function processed the uploaded JSON, the translated output was automatically stored in the response-bucket. This confirmed the end-to-end functionality of the serverless pipeline.

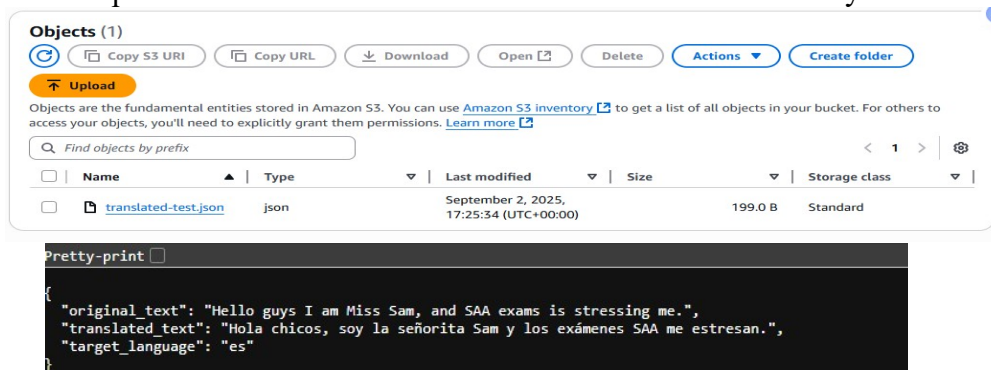


Figure 20 & 21: Translated output file generated by Lambda and saved to S3

PHASE 4: Automation with AWS Lambda Activities

- Packaged the Python script using Terraform's archive_file.
- Deployed AWS Lambda function with:

The translation logic was deployed as an AWS Lambda function using Terraform. This configuration defines the function's runtime, handler, environment variables, and permissions ensuring it integrates seamlessly with the S3 buckets and operates securely.

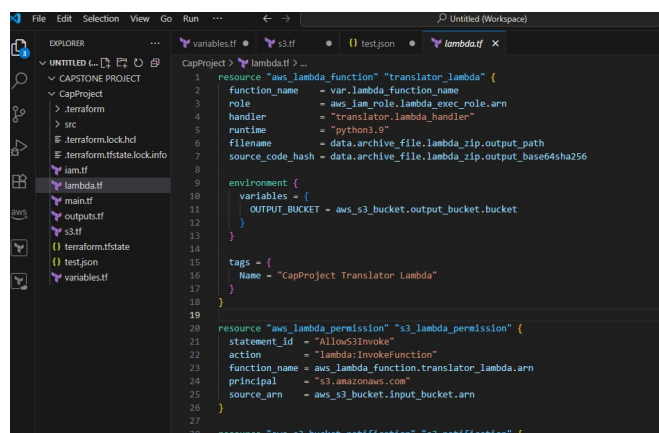


Figure 22: Terraform lambda.tf file defining Lambda function and S3 invocation permissions

- Configured S3 event notification: Whenever a file is uploaded to the input bucket, it triggers Lambda automatically.

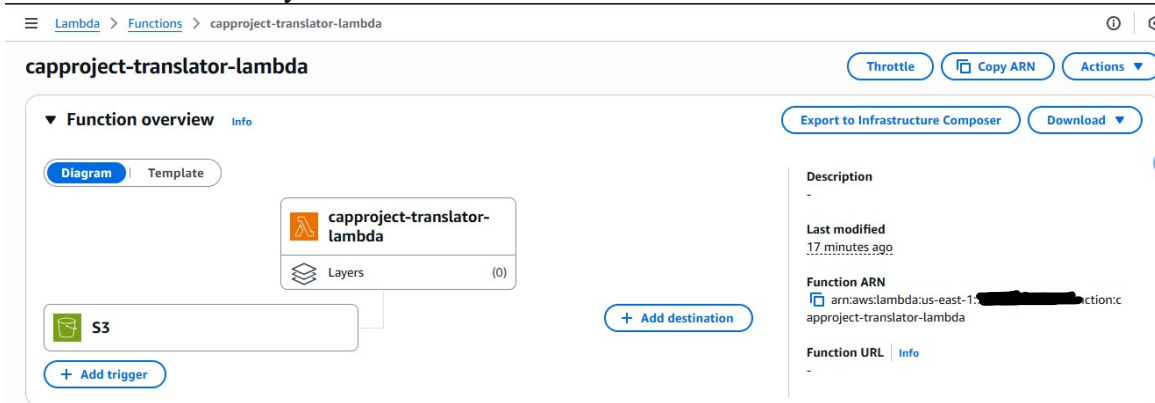


Figure 23 : S3 event notification setup linking file uploads to Lambda invocation

Automated flow:

User uploads JSON → S3 triggers Lambda → Lambda (translator.py) translates → Result stored in response-bucket → CloudWatch logs capture execution details.

PHASE 5: Testing, Debugging & Documentation

ACTIVITIES

- Uploaded sample JSON files to input bucket (refer to fig 18 & 19)
- Verified translation outputs (refer to fig 20 & 21)
- Checked Cloudwatch logs.

After deploying and testing the translation pipeline, CloudWatch Logs were reviewed to validate Lambda execution and identify any errors or performance issues.

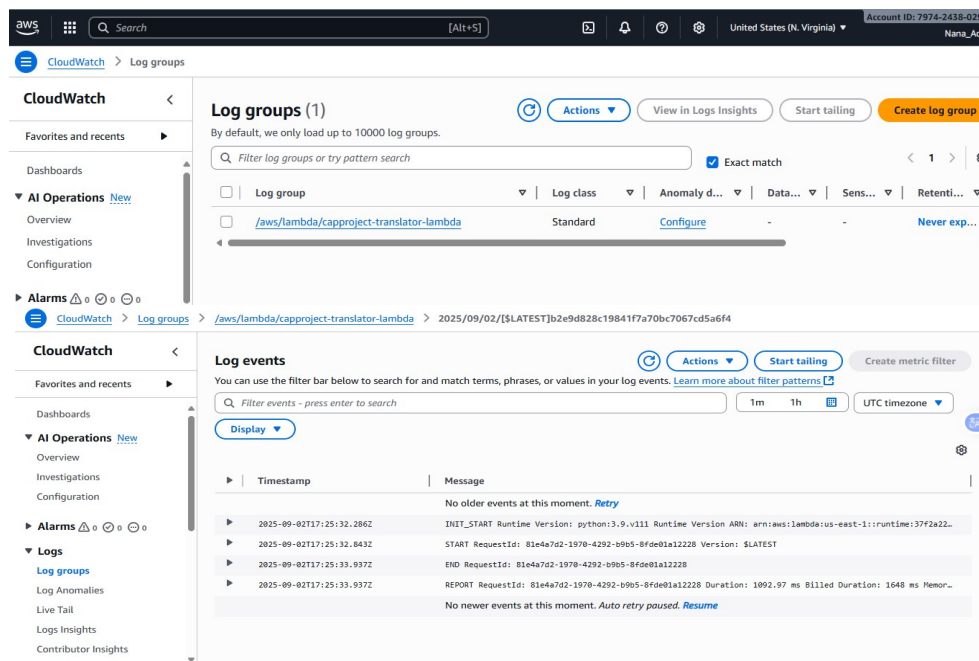


Figure 24: CloudWatch logs showing successful Lambda invocation and translation output

- Finally ran terraform destroy to cleanup resources.

After successful testing and validation, all provisioned AWS resources were cleaned up using Terraform's destroy command. This ensured that no residual infrastructure remained and helped maintain Free Tier compliance.

```
output_bucket_name = "mssam-capstone-bucket-response"
PS C:\Users\euodi\OneDrive\Desktop\CapProject> aws s3 cp test.json s3://mssam-capstone-bucket-request/uploads/test.json
upload: .\test.json to s3://mssam-capstone-bucket-request/uploads/test.json
PS C:\Users\euodi\OneDrive\Desktop\CapProject> terraform destroy
data.archive_file.lambda_zip: Reading...
data.archive_file.lambda_zip: Read complete after 0s [id=arn:aws:lambda:us-east-1:111111111111:lambda:function:lambda_zip]
aws_iam_role.lambda_exec_role: Refreshing state... [id=cappproject-lambda-exec-role]
aws_s3_bucket.input_bucket: Refreshing state... [id=mssam-capstone-bucket-request]
aws_s3_bucket.output_bucket: Refreshing state... [id=mssam-capstone-bucket-response]
aws_s3_bucket_lifecycle_configuration.input_bucket_lifecycle: Refreshing state... [id=mssam-capstone-bucket-request]
aws_iam_policy.lambda_policy: Refreshing state... [id=arn:aws:iam::111111111111:policy/cappproject-lambda-policy]
aws_s3_bucket_lifecycle_configuration.output_bucket_lifecycle: Refreshing state... [id=mssam-capstone-bucket-response]
aws_lambda_function.translator_lambda: Refreshing state... [id=cappproject-translator-lambda]
aws_iam_role_policy_attachment.lambda_policy_attachment: Refreshing state... [id=cappproject-lambda-exec-role-policy-attachment]
aws_lambda_permission.s3_lambda_permission: Refreshing state... [id=AllowS3Invoke]
aws_s3_bucket_notification.s3_notification: Refreshing state... [id=mssam-capstone-bucket-request]
```

Figure 25: Terraform CLI output showing successful execution of terraform destroy

GITHUB REPOSITORY SETUP

GitHub Repository Initialization

To enable version control and collaboration, the project was initialized as a Git repository and connected to GitHub. This ensures that all code, Terraform configurations, and documentation are tracked, backed up, and shareable.

```
euodi@DESKTOP-NUSD162 MINGW64 ~/OneDrive/Desktop/CapProject (master)
$ git add .
euodi@DESKTOP-NUSD162 MINGW64 ~/OneDrive/Desktop/CapProject (master)
$ git init
Reinitialized existing Git repository in C:/Users/euodi/OneDrive/Desktop/CapProject/.git/
euodi@DESKTOP-NUSD162 MINGW64 ~/OneDrive/Desktop/CapProject (master)
$ git add .
euodi@DESKTOP-NUSD162 MINGW64 ~/OneDrive/Desktop/CapProject (master)
```

Figure 26: Git Bash terminal showing git init and git add . commands executed in the project directory

GitHub Commit and Push Confirmation

After initializing the local Git repository and staging all project files, the first commit was made and pushed to GitHub. This step officially published the project to a remote repository, enabling version control, collaboration, and portfolio visibility.

```
euodi@DESKTOP-NUSD162 MINGW64 ~/OneDrive/Desktop/CapProject (main)
$ git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/MsSam08/Capstone-Project.git
git push -u origin main
[main (root-commit) c51623a] first commit
8 files changed, 296 insertions(+)
create mode 100644 iam.tf
create mode 100644 lambda.tf
create mode 100644 main.tf
create mode 100644 outputs.tf
create mode 100644 s3.tf
create mode 100644 src/translator.py
create mode 100644 test.json
create mode 100644 variables.tf
error: remote origin already exists.
info: please complete authentication in your browser...
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 3.22 KiB | 235.00 KiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MsSam08/Capstone-Project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 27: Git Bash terminal confirming initial commit and push to Capstone-Project repository on GitHub

GitHub Repository View

After initializing the local Git repository and pushing the first commit, the project was successfully published to GitHub under the repository name **Capstone-Project**. This serves as the central hub for all code, infrastructure definitions, and documentation

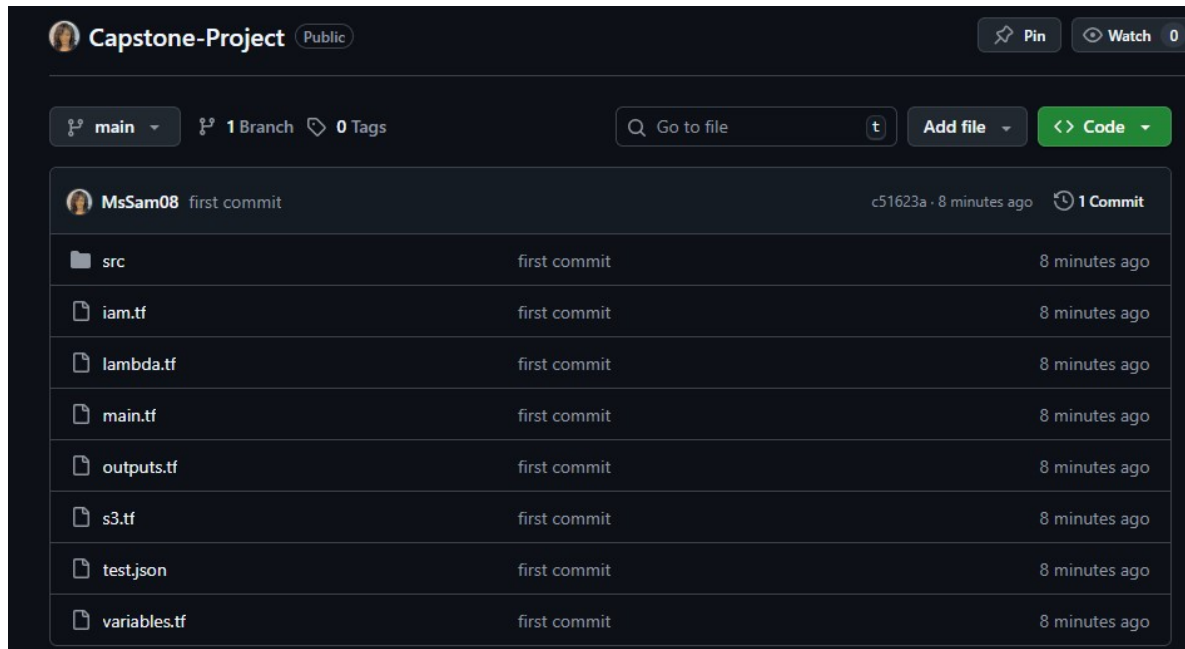


Figure 23: GitHub repository Capstone-Project with Terraform and Python files organized for cloud deployment

FINAL OUTCOME

The project successfully delivered a fully serverless translation pipeline that:

- Automatically translates JSON files uploaded to S3 into the desired target language.
- Stores both original and translated files securely in S3.
- Detects the source language automatically using Amazon Comprehend.
- Logs all execution details and errors in CloudWatch for monitoring and debugging.
- Is reproducibly deployed using Terraform, ensuring consistent infrastructure setup.
- Operates within AWS Free Tier limits for small-scale testing.
- Version-Controlled Deployment:
The entire codebase and infrastructure configuration are tracked in a public GitHub repository, supporting collaboration, documentation, and portfolio visibility.

The final system demonstrates a scalable, cost-efficient, and automated solution for real-time language translation, showcasing best practices in serverless architecture, IaC, and cloud-native development.